

Module 1: Version Control Systems

Lesson Objectives:

- By the end of this lesson, learners will understand the purpose and importance of version control systems, including how they track changes, support collaboration through branching, maintain version history, and ensure teams consistently work with the most up-to-date and accurate files.

Lesson 1.1: What is Version Control?

Version control is the systematic process of tracking and managing updates to source code and assets. Version control systems are foundational for efficient workflows, acting as the single source of truth for programmers while allowing everyone to contribute to a shared codebase independently. They also act as a safety net, making it possible to “undo” code updates if a catastrophic error is introduced during development. Version control also supports the ability to create branches. A branch is a single copy of code managed within a VCS. Developers and other team members can operate independently within distinct streams by establishing branches. In a shared codebase, version control facilitates developers' cooperation and coordination.

Lesson 1.2: Why is version control important?

In today's business environment, the ability to identify what has changed between document versions and make sure users are always distributing or beginning a new revision cycle with the most current one is essential. Document control ensures there is only one master version of each document and that there is a full version history for it.



Implementing a version control system eliminates the time spent manually opening documents to try to determine which one is the most current. You'll also avoid incidentally offering or receiving feedback on the wrong document.

Practice Exercises

- ❖ What is version control, and why is it essential in software development?
- ❖ In what way does version control act as a safety net for developers?
- ❖ What is document control, and how does it relate to version control systems?
- ❖ What problems can arise from not using version control when distributing documents?
- ❖ Why is it important for users to begin a new revision cycle with the most current version of a document?

References

- <https://start.docuware.com/blog/document-management/what-is-version-control-why-is-it-important>
- <https://unity.com/topics/what-is-version-control>
- <https://medium.com/nerd-for-tech/understanding-git-and-version-control-systems-e76da5ec8b34>
- <https://book.the-turing-way.org/reproducible-research/vcs>

Module 2: Introduction to Git

Lesson Objectives:

- By the end of this lesson, learners will understand how Git, a Distributed Version Control System (DVCS), works to manage and track changes in collaborative projects. They will be able to identify key Git concepts—such as repositories, commits, branches, and remote operations.

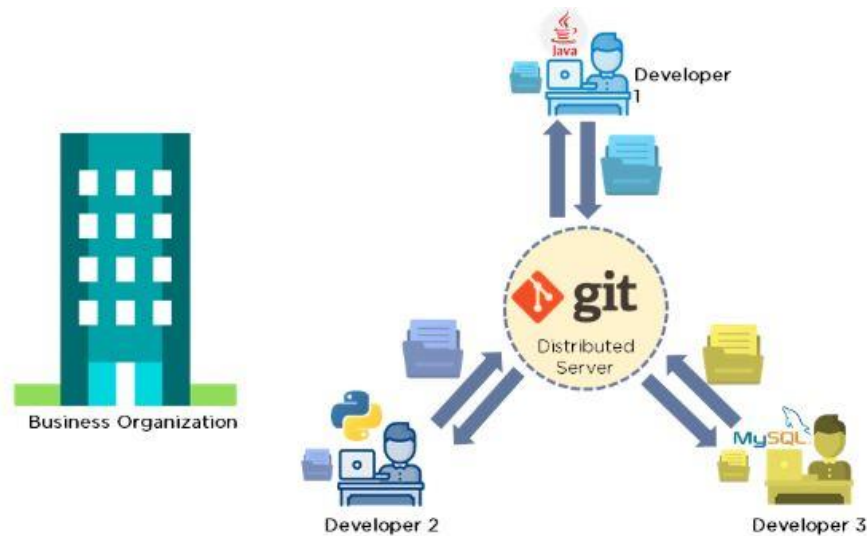
Lesson 2.1: What is Git?

Git is an example of a DVCS (hence Distributed Version Control System). Rather than have only one single place for the full version history of the software as is common in once-popular version control systems like CVS or Subversion (also known as SVN), in Git, every developer's working copy of the code is also a repository that can contain the full history of all changes.

Git is More Than a Software Development Tool

Git can be used for any sort of collaborative project where version control matters, for instance, the writing of a large user manual or even the creation of church music (the last one is a real project that you can view on [GitHub](#))

Although primarily associated with the nuts and bolts coding of software development, people in related fields use Git regularly. Data scientists and analysts are a case in point; these professionals need a way to manage the code that supports their work, and Git provides just that.

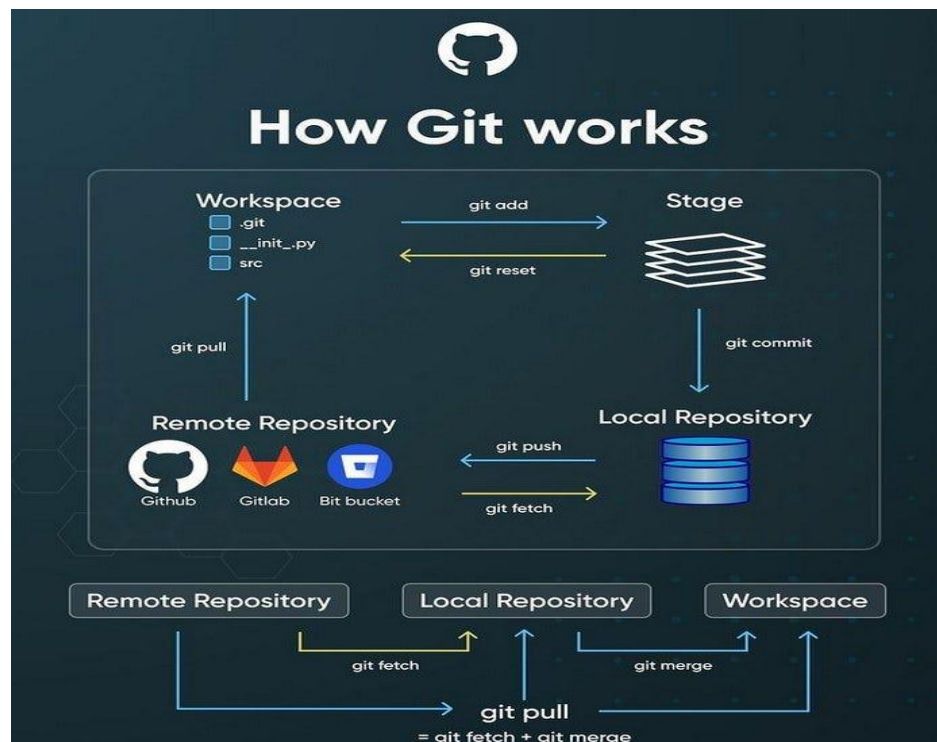


Lesson 2.2: How Does Git Work?

Here's a breakdown of how Git operates:

- **Repository (Repo):** A folder that holds your project and its full history. Created with `git init`.
- **Commit:** A saved snapshot of your changes. Each one has a unique ID.
- **Staging Area:** A prep zone for changes before committing. Add files with `git add`.
- **Branch:** A separate line of development. Use (`git branch <branch-name>`) to create a new branch. The default branch is called `master` or `main`.
- **Merge:** Combines changes from one branch into another using `git merge`.
- **Remote Repo:** An online version of your repo (e.g., on GitHub). Connect with `git remote`.
- **Push/Pull:**
 - `git push` – sends your changes to a remote repo.
 - `git pull` – brings in updates from a remote repo and merges them.
- **Fetch:** Gets changes from a remote repo without merging them yet (`git fetch`).
- **Clone:** Copies a remote repo to your computer (`git clone`).
- **Conflicts:** Happen when changes clash. Git shows these so you can fix them manually.

- **Log:** Git provides with `git log` command to help view commit history.



Practice Exercises

- ❖ What does DVCS stand for, and how does it differ from traditional version control systems like CVS or SVN?
- ❖ Besides software development, what are some other types of collaborative projects where Git can be effectively used?
- ❖ Define what a "Repository" (Repo) is in Git. How is it created?
- ❖ What is a branch in Git, and how can you create a new one?
- ❖ How is the `git fetch` command different from `git pull`?

References

- <https://medium.com/@sujanaddy98/how-git-works-a-visual-guide-with-code-b4edf2694298>
- <https://www.datacamp.com/blog/all-about-git>

Module 3: Installing and Configuring Git

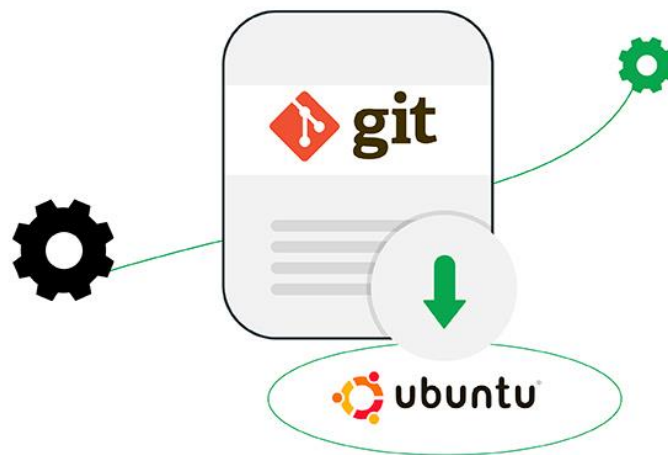
Lesson Objectives:

- By the end of this lesson, learners will be able to determine the proper ways to install Git on Windows, macOS, and Linux computers using package managers or official installers, confirm that the installation was successful using the command line, and understand how crucial it is to keep an updated Git version for development.

Lesson 3.1: Getting Started - Installing Git

You must first make Git available on your computer before you can begin using it. Updating to the most recent version is definitely a good idea, even if it's already installed. You can download the source code and assemble it yourself, or you can install it as a package or through another installer.

Installing Git on Linux



If you want to install the basic Git tools on Linux via a binary installer, you can generally do so through the package management tool that comes with your distribution. To see if you already have Git installed, open up your terminal application.

- If you're on a Windows machine, open the windows command prompt or "Git Bash".
- If you're on a Mac, look for a command prompt application called "Terminal".

and enter:

```
$git --version
```

The output will either tell you which version of Git is installed, or it will alert you that git is an unknown command. If it's an unknown command, read further and find out how to install Git.

If you're on a Debian-based distribution, such as Ubuntu, try apt:

```
$ sudo apt install git-all
```

For more options, there are instructions for installing on several different Unix distributions on the Git website, at <https://git-scm.com/download/linux>.

Installing Git on Windows



There are also a few ways to install Git on Windows. The most official build is available for download on the Git website. Just go to <https://git-scm.com/download/win> and the download will start automatically. To guarantee that Git downloads successfully on your Windows computer, follow these steps:

- Once the installer has started, follow the instructions as provided in the **Git Setup** wizard screen until the installation is complete.
- Open the windows command prompt (or **Git Bash** if you selected not to use the standard Git Windows Command Prompt during the Git installation).
- Type `git version` to verify Git was installed.

To get an automated installation you can use the [Git Chocolatey package](#). Note that the Chocolatey package is community maintained.

Installing Git on Mac



Most versions of MacOS will already have Git installed, and you can activate it through the terminal with `git version`. However, if you don't have Git installed, you can install the latest version of Git using several popular methods as listed below:

Install Git From an Installer

1. Download the most recent version by going to the macOS Git Installer.
2. After the installation has begun, follow the given instructions until the installation is finished.
3. To confirm that Git was installed, open the "terminal" command prompt and enter `git version` to verify Git was installed.

Another popular and recommended resource for downloading Git on a Mac is `git-scm`. `git-scm` has the benefit of automatically starting your download with the most recent version of Git.

Practice Exercises

- ❖ What is the first step you need to take before using Git on your computer?
- ❖ How can you check if Git is already installed on your system? Provide the command that should be used.
- ❖ What command would you use to install Git on a Debian-based Linux distribution like Ubuntu?
- ❖ What are two methods mentioned for installing Git on macOS?
- ❖ After installing Git, what command should you run to verify that the installation was successful?

References

- <https://git-scm.com/book/en/v2/Getting-Started-Installing-Git>
- <https://www.atlassian.com/git/tutorials/install-git>
- <https://github.com/git-guides/install-git>
- <https://www.google.com/url?sa=i&url=https%3A%2F%2Fblog.cloudsigna.com%2Fhow-to-install-and-configure-git-on-ubuntu-18-04%2F&psig=AOvVaw271SGS9k9X5hbWmXuGYqQv&ust=1744302897842000&source=images&cd=vfe&opi=89978449&ved=0CBcQjhqxqFwoTCNDlouw4wDFQAAAAAdAAAAABAQ>
- <https://www.google.com/url?sa=i&url=https%3A%2F%2Fprince-adimado.hashnode.dev%2Fintroduction-to-git-how-to-install-git-configure-and-use-it-on-windows&psig=AOvVaw271SGS9k9X5hbWmXuGYqQv&ust=1744302897842000&source=images&cd=vfe&opi=89978449&ved=0CBcQjhqxqFwoTCNDlouw4wDFQAAAAAdAAAAABAc>
- <https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.igeeksblog.com%2Fhow-to-install-git-on-mac%2F&psig=AOvVaw3KoFfoV2YFVnPys7Y76UMI&ust=1744303063603000&source=images&cd=vfe&opi=89978449&ved=0CBcQjhqxqFwoTCPC9nbqxy4wDFQAAAAAdAAAAABAE>

Module 4: Basic Git Commands

Lesson Objectives:

- By the end of this lesson, learners will be able to understand how to execute basic Git commands to initialize a repository, track changes, commit changes with meaningful messages to maintain a clear project history and utilize commands for pushing and pulling updates to and from remote repositories.

Lesson 4.1: Setting up Git

The first thing you should do when you install Git is to set your username and email address. This is important because every Git commit uses this information. After installing Git on your local computer, here are the steps to configuring Git for the first time:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "you@example.com"
```


then view your configuration using this command:

```
git config --list
```

You need to do this only once if you pass the `--global` option, because then Git will always use that information for anything you do on that system. If you want to override this with a different name or email address for specific projects, you can run the command without the `--global` option when you're in that project.

Lesson 4.2: Creating a Git Repository

What is a Git Repository?

A [Git repository](#) is a virtual storage of your project. It allows you to save versions of your code, which you can access when needed.

Creating a Git Repository

The `git init` command is used to create a new repository. When you first set up a new repository, you execute the one-time command `git init`. A new `.git` subdirectory will be created in your current working directory when you run this command. Additionally, a new `main` branch will be created.

```
git init
```

Cloning an existing repository: `git clone`

The `clone` command is the most popular method for users to get a local development clone of a project that has previously been set up in a central repository. Cloning is typically a one-time process, much like `git init`. All version control functions are controlled by the developer's local repository after they have a functioning copy.

To clone an existing repository, use the following command:

```
git clone <repo url>
```

Tracking Changes

This allows developers to keep an eye on and control changes made to project files. Git does not automatically add changes you make to your files to your version history. Instead, you use `git add` to stage the changes you want to include in the next snapshot, and then use `git commit` to save those changes with a descriptive message.

To check the repository status use this command:

```
git status
```

To add files to staging use this command:

```
git add filename.txt
```

To commit changes use this command:

```
git commit -m "Commit message"
```

Repo-to-repo collaboration: git push

The foundation of Git's collaboration approach is repository-to-repository communication. You push or pull commits from one repository to another rather than checking a working copy into SVN's central repository.

You won't have a remote repository to push changes to if you created a new repository using git init. Creating a repository on a hosted Git service, such as Bitbucket, is a typical practice when starting a new repository. After receiving a Git URL from the service, you can add it to your local repository and push changes to the hosted repository. You must update your local repository with a mapping after creating a remote repository with your preferred service.

After setting up a remote repository, you must create an upstream branch for your local branches and add a remote repository URL to your local git configuration. One such tool is provided by the git remote command.

```
git remote add <remote_name> <remote_repo_url>
```

This command will map a reference in your local repository under <remote_name> to a remote repository at <remote_repo_url>. You can push local branches to the remote repository after mapping it.

```
git push -u <remote_name> <local_branch_name>
```

To pull changes use this command:

```
git pull -u <remote_name>
```

Here is a Git Cheat Sheet that includes the basic Git commands to help you learn Git, and more advanced concepts around Git branches, remote repositories, undoing changes, and more.

Git Cheat Sheet



GIT BASICS

<code>git init <directory></code>	Create empty Git repo in specified directory. Run with no arguments to initialize the current directory as a git repository.
<code>git clone <repo></code>	Clone repo located at <repo> onto local machine. Original repo can be located on the local filesystem or on a remote machine via HTTP or SSH.
<code>git config user.name <name></code>	Define author name to be used for all commits in current repo. Devs commonly use <code>--global</code> flag to set config options for current user.
<code>git add <directory></code>	Stage all changes in <directory> for the next commit. Replace <directory> with a <file> to change a specific file.
<code>git commit -m "<message>"</code>	Commit the staged snapshot, but instead of launching a text editor, use <message> as the commit message.
<code>git status</code>	List which files are staged, unstaged, and untracked.
<code>git log</code>	Display the entire commit history using the default format. For customization see additional options.
<code>git diff</code>	Show unstaged changes between your index and working directory.

UNDOING CHANGES

<code>git revert <commit></code>	Create new commit that undoes all of the changes made in <commit>, then apply it to the current branch.
<code>git reset <file></code>	Remove <file> from the staging area, but leave the working directory unchanged. This unstages a file without overwriting any changes.
<code>git clean -n</code>	Shows which files would be removed from working directory. Use the <code>-f</code> flag in place of the <code>-n</code> flag to execute the clean.

REWRITING GIT HISTORY

<code>git commit --amend</code>	Replace the last commit with the staged changes and last commit combined. Use with nothing staged to edit the last commit's message.
<code>git rebase <base></code>	Rebase the current branch onto <base>. <base> can be a commit ID, branch name, a tag, or a relative reference to HEAD.
<code>git reflog</code>	Show a log of changes to the local repository's HEAD. Add <code>--relative-date</code> flag to show date info or <code>--all</code> to show all refs.

GIT BRANCHES

<code>git branch</code>	List all of the branches in your repo. Add a <branch> argument to create a new branch with the name <branch>.
<code>git checkout -b <branch></code>	Create and check out a new branch named <branch>. Drop the <code>-b</code> flag to checkout an existing branch.
<code>git merge <branch></code>	Merge <branch> into the current branch.

REMOTE REPOSITORIES

<code>git remote add <name> <url></code>	Create a new connection to a remote repo. After adding a remote, you can use <name> as a shortcut for <url> in other commands.
<code>git fetch <remote> <branch></code>	Fetches a specific <branch>, from the repo. Leave off <branch> to fetch all remote refs.
<code>git pull <remote></code>	Fetch the specified remote's copy of current branch and immediately merge it into the local copy.
<code>git push <remote> <branch></code>	Push the branch to <remote>, along with necessary commits and objects. Creates named branch in the remote repo if it doesn't exist.

Additional Options +

GIT CONFIG

<code>git config --global user.name <name></code>	Define the author name to be used for all commits by the current user.
<code>git config --global user.email <email></code>	Define the author email to be used for all commits by the current user.
<code>git config --global alias.<alias-name> <git-command></code>	Create shortcut for a Git command. E.g. <code>alias.glog "log --graph --oneline"</code> will set "git glog" equivalent to "git log --graph --oneline".
<code>git config --system core.editor <editor></code>	Set text editor used by commands for all users on the machine. <editor> arg should be the command that launches the desired editor (e.g., vi).
<code>git config --global --edit</code>	Open the global configuration file in a text editor for manual editing.

GIT LOG

<code>git log --<limit></code>	Limit number of commits by <limit>. E.g. "git log -5" will limit to 5 commits.
<code>git log --oneline</code>	Condense each commit to a single line.
<code>git log -p</code>	Display the full diff of each commit.
<code>git log --stat</code>	Include which files were altered and the relative number of lines that were added or deleted from each of them.
<code>git log --author="<pattern>"</code>	Search for commits by a particular author.
<code>git log --grep="<pattern>"</code>	Search for commits with a commit message that matches <pattern>.
<code>git log <since>..<until></code>	Show commits that occur between <since> and <until>. Args can be a commit ID, branch name, HEAD, or any other kind of revision reference.
<code>git log -- <file></code>	Only display commits that have the specified file.
<code>git log --graph --decorate</code>	--graph flag draws a text based graph of commits on left side of commit msg. --decorate adds names of branches or tags of commits shown.

GIT DIFF

<code>git diff HEAD</code>	Show difference between working directory and last commit.
<code>git diff --cached</code>	Show difference between staged changes and last commit

GIT RESET

<code>git reset</code>	Reset staging area to match most recent commit, but leave the working directory unchanged.
<code>git reset --hard</code>	Reset staging area and working directory to match most recent commit and overwrites all changes in the working directory.
<code>git reset <commit></code>	Move the current branch tip backward to <commit>, reset the staging area to match, but leave the working directory alone.
<code>git reset --hard <commit></code>	Same as previous, but resets both the staging area & working directory to match. Deletes uncommitted changes, and all commits after <commit>.

GIT REBASE

<code>git rebase -i <base></code>	Interactively rebase current branch onto <base>. Launches editor to enter commands for how each commit will be transferred to the new base.
---	---

GIT PULL

<code>git pull --rebase <remote></code>	Fetch the remote's copy of current branch and rebases it into the local copy. Uses git rebase instead of merge to integrate the branches.
---	---

GIT PUSH

<code>git push <remote> --force</code>	Forces the git push even if it results in a non-fast-forward merge. Do not use the --force flag unless you're absolutely sure you know what you're doing.
<code>git push <remote> --all</code>	Push all of your local branches to the specified remote.
<code>git push <remote> --tags</code>	Tags aren't automatically pushed when you push a branch or use the --all flag. The --tags flag sends all of your local tags to the remote repo.



Visit atlassian.com/git for more information, training, and tutorials

Practice Exercises

- ❖ Initialize a Git repository and make your first commit.
- ❖ Modify a file and practice staging and committing the changes.
- ❖ Use git log to review history.
- ❖ Try undoing a change using git checkout and git revert.
- ❖ Set up a GitHub repository and push your local repo to it.

For more detail, check out the [Atlassian Git Tutorials](https://confluence.atlassian.com/bitbucketserver/basic-git-commands-776639767.html) for a visual introduction to Git commands and workflows, including examples.

References

<https://confluence.atlassian.com/bitbucketserver/basic-git-commands-776639767.html>

<https://www.atlassian.com/git/tutorials/atlassian-git-cheatsheet>

<https://www.atlassian.com/git/tutorials/setting-up-a-repository>

https://www.atlassian.com/git?utm_source=basic-git-commands&utm_medium=link&utm_campaign=git-microsite